

13. BLOB Operation

A **BLOB** (Binary Large Object) is used to store binary data, such as images, videos, or other multimedia objects, in a database. In JDBC, we can use PreparedStatement for inserting BLOB data and ResultSet for retrieving it.

Signature of setBinaryStream method

The syntax of setBinaryStream() method is given below:

- 1) public void setBinaryStream(int paramIndex,InputStream stream) throws SQLException
- 2) public void setBinaryStream(int paramIndex,InputStream stream,long length) throws SQLException

Database Table Creation

Run the following SQL query to create the required table:

```
CREATE TABLE BlobExample (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(255),  
    file_data LONGBLOB  
);
```

Example to store the blob content in Database:

```
package com.lokesh;  
  
import java.io.*;  
import java.sql.*;  
  
public class BlobStoreExample {  
  
    public static void main(String[] args) {  
        String jdbcUrl = "jdbc:mysql://localhost:3306/organization";  
        String username = "root";  
        String password = "root";  
  
        Connection connection = null;  
        PreparedStatement preparedStatement = null;  
  
        try {  
            // Establish connection  
            connection = DriverManager.getConnection(jdbcUrl, username, password);  
  
            // Read the file from the resources folder  
            File file = new File("src/main/resources/store_image.jpg");  
            FileInputStream inputStream = new FileInputStream(file);  
  
            // Prepare the SQL statement  
            String sql = "INSERT INTO BlobExample (name, file_data) VALUES (?, ?)";  
            preparedStatement = connection.prepareStatement(sql);  
            preparedStatement.setString(1, file.getName());  
            preparedStatement.setBinaryStream(2, inputStream, (int) file.length());  
  
            // Execute the query  
            int rowsInserted = preparedStatement.executeUpdate();  
        }  
    }  
}
```

```

        if (rowsInserted > 0) {
            System.out.println("File successfully stored as BLOB.");
        }

        inputStream.close();
    } catch (Exception e) {
        e.printStackTrace();
    } finally {
        // Close resources
        try {
            if (preparedStatement != null) preparedStatement.close();
            if (connection != null) connection.close();
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
}
}

```

Output:

```
File successfully stored as BLOB.
```

By the help of PreparedStatement we can retrieve and store the image in the database. The getBlob() method of PreparedStatement is used to get Binary information, it returns the instance of Blob. After calling the getBytes() method on the blob object, we can get the array of binary information that can be written into the image file.

Signature of getBlob() method of PreparedStatement

public Blob getBlob() throws SQLException

Signature of getBytes() method of Blob interface

public byte[] getBytes(long pos, int length) throws SQLException

Example to retrieve the blob content in Database:

```

package com.lokesh;

import java.io.*;
import java.sql.*;

public class BlobRetrieveExample {

    public static void main(String[] args) {
        String jdbcUrl = "jdbc:mysql://localhost:3306/organization";
        String username = "root";
        String password = "root";

        Connection connection = null;
        PreparedStatement preparedStatement = null;
        ResultSet resultSet = null;

        try {
            // Establish connection
            connection = DriverManager.getConnection(jdbcUrl, username, password);

            // Prepare the SQL query
            String sql = "SELECT name, file_data FROM BlobExample WHERE id = ?";

```

```
preparedStatement = connection.prepareStatement(sql);
preparedStatement.setInt(1, 1); // Assuming the ID of the file to retrieve is 1

// Execute the query
resultSet = preparedStatement.executeQuery();

if (resultSet.next()) {
    String fileName = resultSet.getString("name");
    InputStream inputStream = resultSet.getBinaryStream("file_data");

    // Save the file to the resources folder
    File outputFile = new File("src/main/resources/retrieved_" + fileName);
    FileOutputStream outputStream = new FileOutputStream(outputFile);

    byte[] buffer = new byte[1024];
    int bytesRead;
    while ((bytesRead = inputStream.read(buffer)) != -1) {
        outputStream.write(buffer, 0, bytesRead);
    }

    System.out.println("File successfully retrieved and saved to resources
                                                                folder.");

    outputStream.close();
    inputStream.close();
}
} catch (Exception e) {
    e.printStackTrace();
} finally {
    // Close resources
    try {
        if (resultSet != null) resultSet.close();
        if (preparedStatement != null) preparedStatement.close();
        if (connection != null) connection.close();
    } catch (SQLException e) {
        e.printStackTrace();
    }
}
}
```

Output:

```
File successfully retrieved and saved to resources folder.
```